

Guide to Burning Bootloader on ATmega2560 and Flashing Firmware on ATmega16U2

Ian Costa

July 8, 2025

Contents

1	Introduction	2
2	Environment Setup	2
3	Burning the Bootloader onto ATmega2560	3
3.1	Setting Up Programmer	3
3.2	Connections	3
3.3	Burning the Bootloader	6
4	Flashing Firmware onto ATmega16U2	7
4.1	Connections	7
4.2	Command Setup	8
4.3	Setting Fuses	9
4.4	Flashing the Firmware Hex File	9

1 Introduction

This document describes the step-by-step procedure for:

- Burning the bootloader onto the ATmega2560 microcontroller
- Flashing USB-Serial firmware onto the ATmega16U2

Hardware Needed:

- AVR programmer (e.g., USBtiny)
- Arduino Uno with USB cable to connect to computer
- ATmega2560 with ICSP connection available and clock circuitry
- ATmega16u2 with ICSP connection available and clock circuitry
- Jumper wires

2 Environment Setup

1. Install Required Software

- AVRDUDE
- Arduino IDE

3 Burning the Bootloader onto ATmega2560

3.1 Setting Up Programmer

In order to burn the bootloader onto an ATmega2560, you will need to open the Arduino IDE program and first download the ISP sketch onto the Arduino Uno we will be using to use as the programmer.

- Open Arduino IDE
- Connect Arduino to the computer
- Select your programmer board in:

`Tools > Board > Arduino AVR Boards > Arduino Uno`

- Select port the board is connected to in:

`Tools > Port`

(This may require you to check which port it is connected to using device manager)

- Open:

`File > Examples > 11.ArduinoISP > ArduinoISP`

- Click download

3.2 Connections

Unplug the Arduino from the computer. You can now make connections from the Arduino to the ATmega2560. Before this, however, make sure that the required clock circuitry is connected to the chip. This would mean a 16kHz oscillator across pins 33 and 34 with two 22pF capacitors in series across the oscillator as well.

- Connect jumper cables to the ICSP port pins on the Arduino while noting each pin's function **except for the RESET pin** (you will need to make one-to-one connections to match from the Arduino to the chip):



Figure 1: Arduino ICSP pins

- Connect your RESET jumper cable instead to pin 10 on the Arduino (not on the ICSP header).
- You will now make your connections to the chip following this diagram:

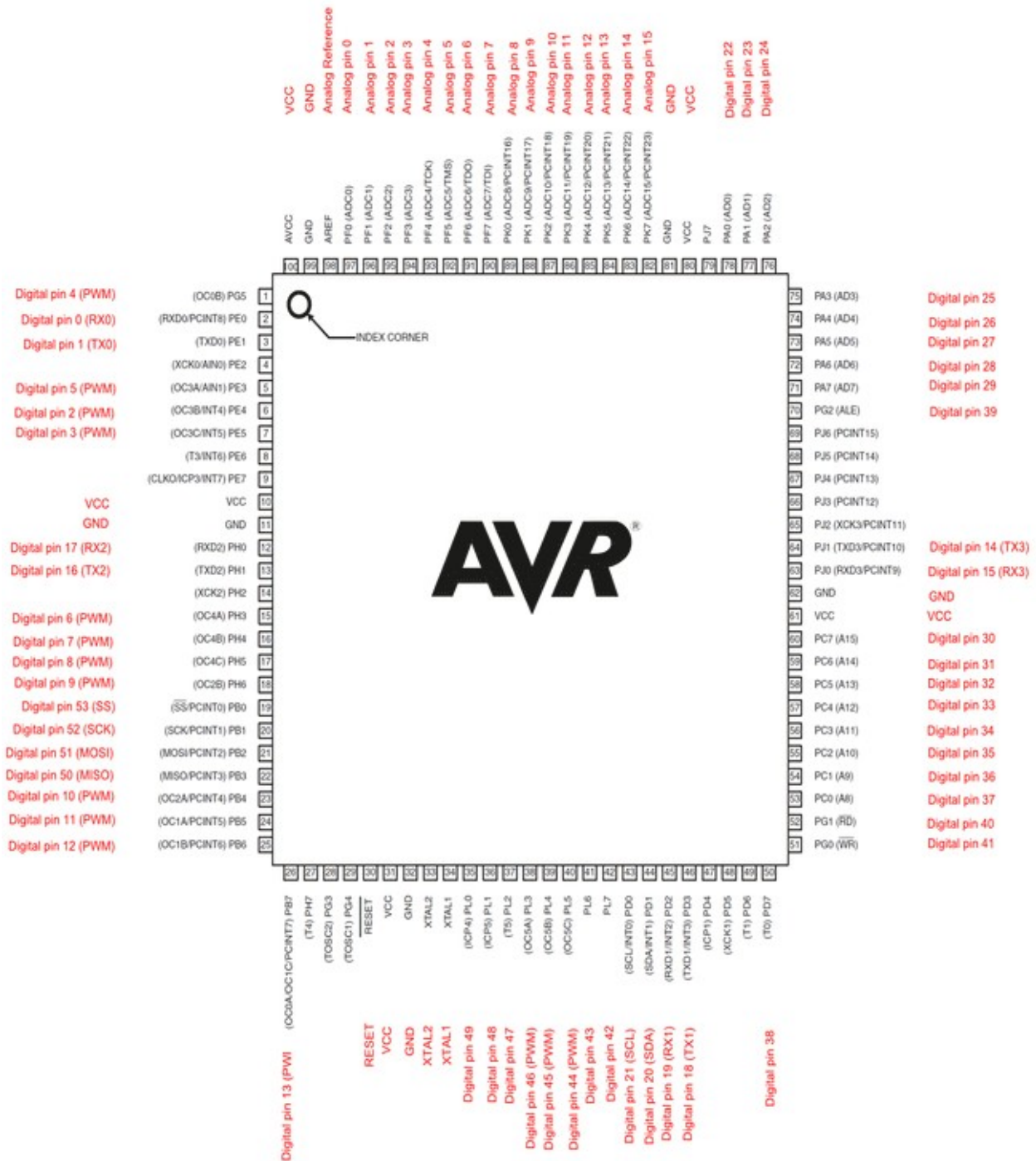


Figure 2: ATmega2560 Pinout

- Put simply, here are the connections to be made in Arduino>2560 format:

- VCC > Pin 10
- GND > Pin 11
- SCK > Pin 20
- MOSI > Pin 21
- MISO > Pin 22
- Pin 10 > Pin 30

3.3 Burning the Bootloader

After making the connections and plugging the Arduino back into the computer, you can begin the final step of burning the bootloader.

- Make sure you have the port ID of the Arduino.
- In Arduino IDE, go to:

Tools > Port

and make sure that the port selected is the correct one.

- Select the target board in:

Tools > Board

and set it to "Arduino Mega or Mega 2560".

- Set the programmer by:

Tools > Programmer > Arduino as ISP

- Finally, select:

Tools > Burn Bootloader

- It should take a few seconds, but you will know it has been successful if you see "Done Burning Bootloader" at the bottom right of the screen.

4 Flashing Firmware onto ATmega16U2

4.1 Connections

- We will begin by making connections from the USBtiny to the ATmega16u2. Again we must ensure the clock circuitry is connected to the chip, this time to pins 1 and 2.
- The pins for the USBtiny are labeled on the back of the board. For the target however is shown in this diagram:

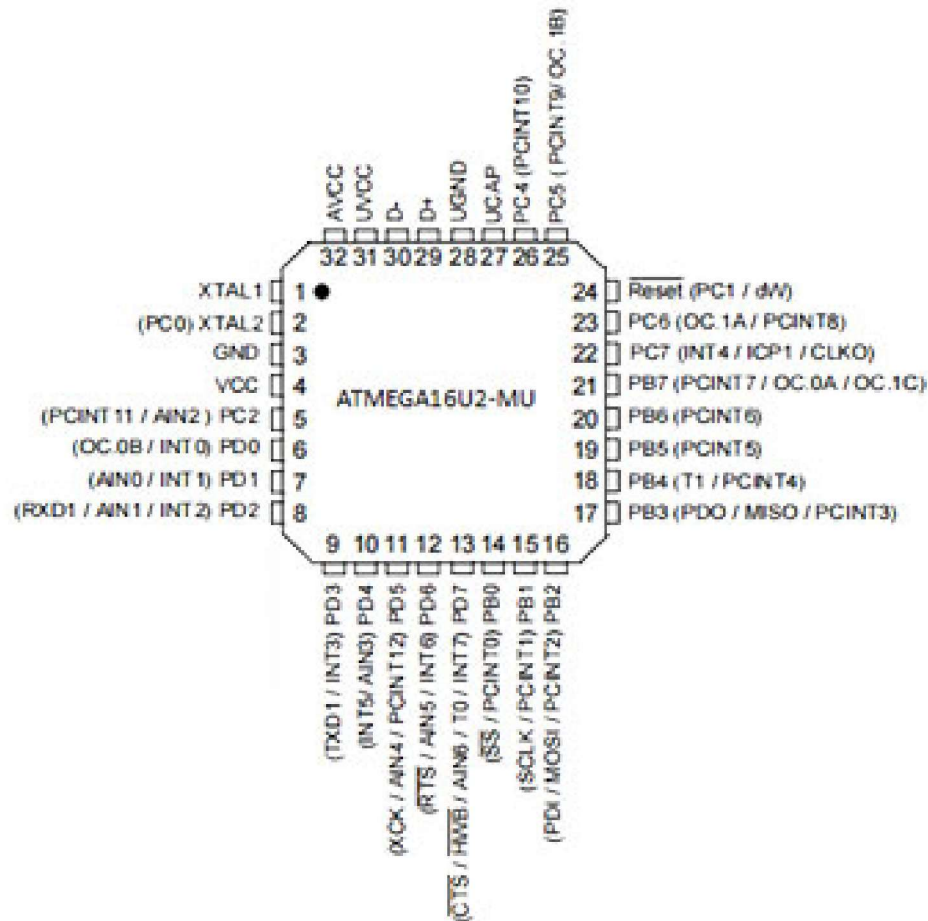


Figure 3: ATmega16u2 Pinout

- Put simply:
 - VCC > Pin 4
 - GND > Pin 3
 - SCK > Pin 15
 - MOSI > Pin 16

- MISO > Pin 17
- RESET > Pin 24
- Before plugging into the computer, make sure that the switch on the USBtiny is set to "Power Target".

4.2 Command Setup

- Open Command Prompt
- Navigate to your avrdude file in the command prompt and enter the avrdude file
- Type:

```
avrdude -p at90usb162 -c -usbtiny -F -t
```

This command works as follows:

- "avrdude" is the program itself that talks to the chip
- "-p at90usb162" is the identity of the chip that we are programming. We cannot use "16u2" because it is not a valid identification in the avrdude program but "162" will work in this instance.
- "-c usbtiny" tells the program which programmer we will be using
- "-F" tells the program to override signature check failures since it will fail due to the difference in signature between the "16u2" and the "162" chips.
- "-t" Puts us into terminal mode.
- Type:

```
erase
```

then:

```
quit
```

once allowed. This erases any previous data on the chip to prime it for the firmware download.

4.3 Setting Fuses

Here we will set the hardware in the chip to facilitate a firmware upload.

- Type:

```
avrdude -p at90usb162 -c usbtiny -F -U efuse:w:0xF4:m -U hfuse:w:0xD9:m  
  
-U lfuse:w:0xFF:m
```

It works like this:

- "hfuse, lfuse, etc." is which fuse we are concerned with.
- "w" tells the program that we will be writing to it as opposed to reading "r".
- "0XF4, etc." is the value we will be writing
- "m" means that it is setting an immediate value

4.4 Flashing the Firmware Hex File

Finally, we will flash the firmware to the chip.

- Type:

```
avrdude -p at90usb162 -c usbtiny -U flash:w:"Firmware Hex Directory Path"
```

You will know it is successfull if by the end the two progress bars have filled and the terminal says "avrdude done"